

Gradient Boosted Trees: Algorithms 10.3 & 10.4

From *The Elements of Statistical Learning* — Hastie, Tibshirani, Friedman

Anuj Rajan Lalla (B22AI061)

Abhinav Swami (B22AI003)

Abstract

We implement Algorithms 10.3 (Gradient Tree Boosting for regression) and 10.4 (Gradient Boosting for K -class classification) from scratch, using only `sklearn.tree.DecisionTreeRegressor` as the base learner. The entire boosting loop of pseudo-residual computation, leaf-value optimization, and shrinkage-based updates is coded manually. We evaluate regression on **California Housing** and **Friedman #1**, and classification on **Digits** ($K=10$) and **Wine** ($K=3$). Systematic ablations over the number of iterations M , tree depth J , learning rate ν , and loss function reveal clear bias–variance tradeoffs. On Digits, gradient boosting reaches **97.04%** test accuracy, substantially outperforming a single stump (18.70%) and a single deep tree (83.33%). Robust loss functions (absolute error, Huber) reduce test MSE by $\sim 4\times$ over squared error when 5% outliers are present.

1 Introduction

Gradient boosting constructs an additive model $\hat{f}(x) = \sum_{m=0}^M T(x; \Theta_m)$ by sequentially fitting regression trees to the *negative gradient* of a loss function evaluated at the current model. Unlike AdaBoost (which uses exponential loss and re-weights samples), gradient boosting generalizes to arbitrary differentiable losses, enabling robust regression (Huber, absolute error) and multi-class classification (multinomial deviance).

We implement two algorithms:

- **Algorithm 10.3:** General gradient tree boosting for regression, supporting squared error, absolute error, and Huber loss.
- **Algorithm 10.4:** Gradient boosting for K -class classification using softmax probabilities and Newton-step leaf values.

2 Algorithm Descriptions

2.1 Algorithm 10.3 : Gradient Tree Boosting (Regression)

1. **Initialize** $f_0(x) = \arg \min_{\gamma} \sum_{i=1}^N L(y_i, \gamma)$.
2. **For** $m = 1$ to M :
 - (a) Compute pseudo-residuals: $r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}}$
 - (b) Fit a regression tree to $\{r_{im}\}$, giving terminal regions R_{jm} .
 - (c) Compute optimal leaf values: $\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$
 - (d) Update: $f_m(x) = f_{m-1}(x) + \nu \sum_j \gamma_{jm} I(x \in R_{jm})$
3. **Output** $\hat{f}(x) = f_M(x)$.

For squared error loss, the pseudo-residuals reduce to the ordinary residuals $y_i - f_{m-1}(x_i)$, and the optimal leaf value is the mean of residuals in each region. For absolute error, the negative gradient is $\text{sign}(y_i - f)$ and the leaf value is the *median*. For Huber loss, the gradient is a hybrid: the residual when $|y_i - f| \leq \delta_m$ and $\delta_m \cdot \text{sign}(y_i - f)$ otherwise, where δ_m is the α -quantile of $|y_i - f_{m-1}(x_i)|$.

2.2 Algorithm 10.4 : Gradient Boosting for K -class Classification

We maintain K additive models $f_k(x)$, one per class, and obtain probabilities via softmax:

$$p_k(x) = \frac{e^{f_k(x)}}{\sum_{\ell=1}^K e^{f_\ell(x)}}$$

1. **Initialize** $f_{k0}(x) = 0$, $k = 1, \dots, K$.
2. **For** $m = 1$ to M :
 - (a) Compute probabilities $p_k(x)$ via softmax.
 - (b) For each class $k = 1, \dots, K$:
 - Compute residuals: $r_{ikm} = y_{ik} - p_k(x_i)$
 - Fit a regression tree to $\{r_{ikm}\}$, giving regions R_{jkm} .
 - Compute leaf values via Newton step:

$$\gamma_{jkm} = \frac{K-1}{K} \cdot \frac{\sum_{x_i \in R_{jkm}} r_{ikm}}{\sum_{x_i \in R_{jkm}} |r_{ikm}|(1 - |r_{ikm}|)}$$

- Update: $f_{km}(x) = f_{k,m-1}(x) + \nu \sum_j \gamma_{jkm} I(x \in R_{jkm})$

3. **Output** $\hat{f}_k(x) = f_{kM}(x)$, $k = 1, \dots, K$.

The $\frac{K-1}{K}$ factor arises from a diagonal-Hessian approximation of the Newton-Raphson update on the log-likelihood within each terminal region, evaluated at $\gamma = 0$.

3 Datasets

Table 1: Datasets used in experiments.

Task	Dataset	Train	Test	Features	Classes/ K
Regression	California Housing	16,512	4,128	8	—
Regression	Friedman #1 (synthetic)	4,000	1,000	10	—
Classification	Digits	1,257	540	64	10
Classification	Wine	124	54	13	3

California Housing: Predicts median house values from 8 census features. A moderately large, real-world regression benchmark.

Friedman #1: The true function is $y = 10 \sin(\pi x_1 x_2) + 20(x_3 - 0.5)^2 + 10x_4 + 5x_5 + \varepsilon$, where only 5 of 10 features are relevant. Useful because the ground truth is known.

Digits: 8×8 grayscale images of handwritten digits (0–9) with 64 features. Substantially harder than 3-class datasets, making it ideal for demonstrating boosting’s advantage over single trees.

Wine: 13 chemical measurements for 3 cultivar classes. A small but well-separated classification task.

4 Results

4.1 Regression

Table 2: Regression results with default hyperparameters ($M=200$, depth=4, $\nu=0.1$, squared error).

Dataset	Train MSE	Test MSE	Train R^2	Test R^2
California Housing	0.1712	0.2378	0.8720	0.8185
Friedman #1	0.6414	1.5384	0.9741	0.9376

Both models show a visible train–test gap, confirming some overfitting that can be mitigated via shrinkage or early stopping. The learning curves below show how train and test MSE evolve across boosting iterations.

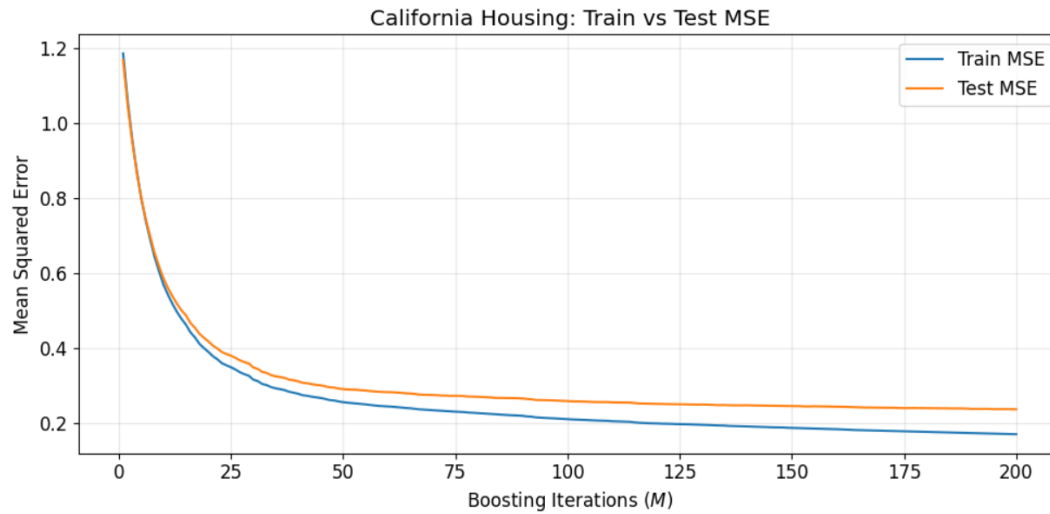


Figure 1: California Housing: Train vs Test MSE across boosting iterations ($M=200$, depth=4, $\nu=0.1$).

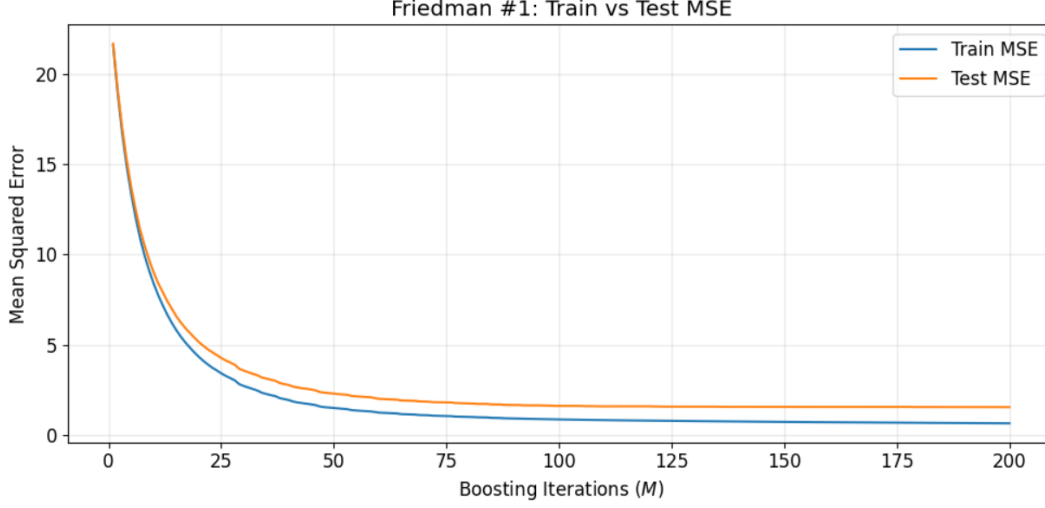


Figure 2: Friedman #1: Train vs Test MSE across boosting iterations ($M=200$, $\text{depth}=4$, $\nu=0.1$).

4.2 Classification

Table 3: Classification results with default hyperparameters ($M=100$, $\text{depth}=3$ for Digits, $\text{depth}=2$ for Wine, $\nu=0.1$).

Dataset	Train Accuracy	Test Accuracy	Macro F1 (Test)
Digits ($K=10$)	1.0000	0.9704	0.97
Wine ($K=3$)	1.0000	0.9815	0.98

Train accuracy of 1.0 on both datasets indicates the boosted ensemble can perfectly memorize the training data. On Digits, per-class F1 ranges from 0.93 (digit 9) to 1.00 (digit 1), with digit 8 showing the highest recall (1.00) but lower precision (0.90) due to confusion with similar-looking digits.



Figure 3: Digits: Train vs Test error rate across boosting iterations.

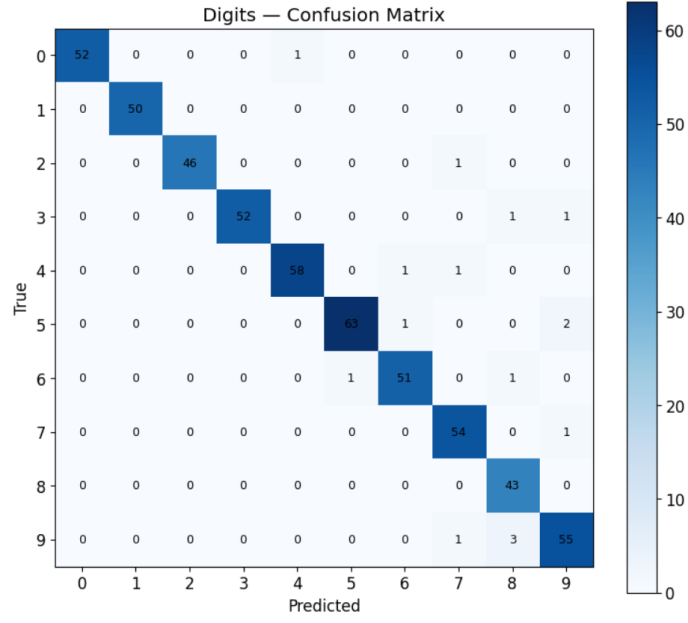


Figure 4: Digits confusion matrix ($K=10$). Most errors involve visually similar digit pairs.

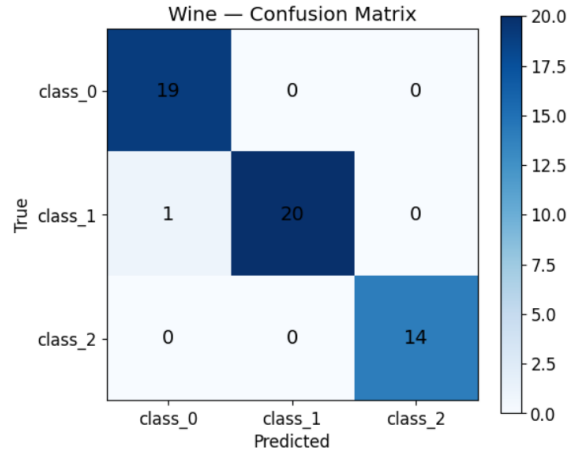


Figure 5: Wine confusion matrix ($K=3$). Near-perfect separation across all three cultivars.

5 Hyperparameter Ablations

5.1 Number of Boosting Iterations M

Table 4: Effect of M on California Housing (depth=4, $\nu=0.1$).

M	Train MSE	Test MSE	Gap
10	0.5699	0.5867	0.017
50	0.2567	0.2910	0.034
100	0.2113	0.2592	0.048
200	0.1712	0.2370	0.066
300	0.1481	0.2283	0.080
500	0.1194	0.2196	0.100

Key observation: The train–test gap widens monotonically with M . At $M=500$, the gap (0.100) is $\sim 6\times$ larger than at $M=10$ (0.017), indicating progressive overfitting. The test MSE continues to decrease but the rate of improvement slows significantly after $M \approx 200$, suggesting diminishing returns.

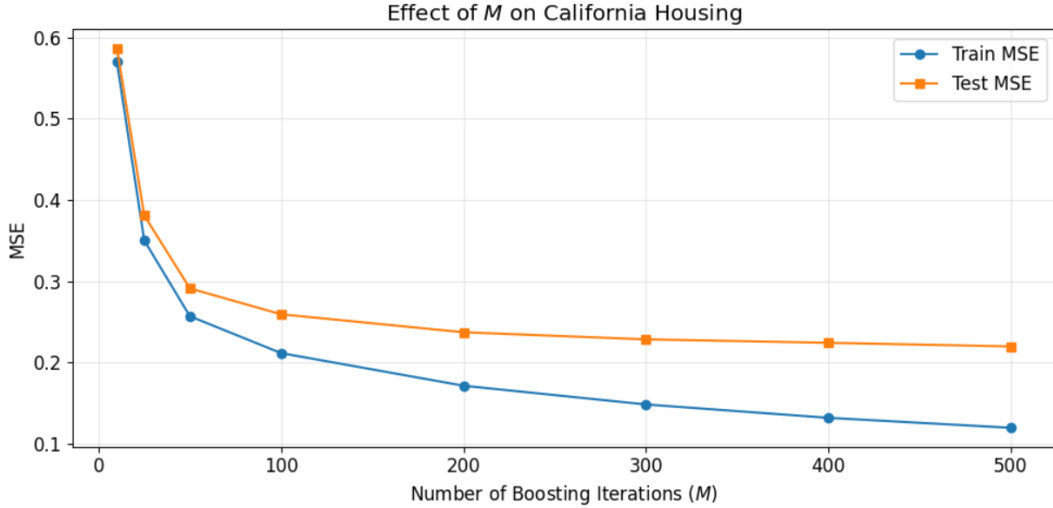


Figure 6: Effect of M on California Housing. The widening gap between train and test MSE indicates overfitting at large M .

For **Digits classification**, train accuracy saturates at 1.0 by $M=50$, while test accuracy improves from 0.8907 ($M=5$) to 0.9741 ($M=150$) and then *plateaus*, then the model stops generalizing further despite continued training. This plateau rather than degradation suggests the learning rate is small enough to provide implicit regularization.

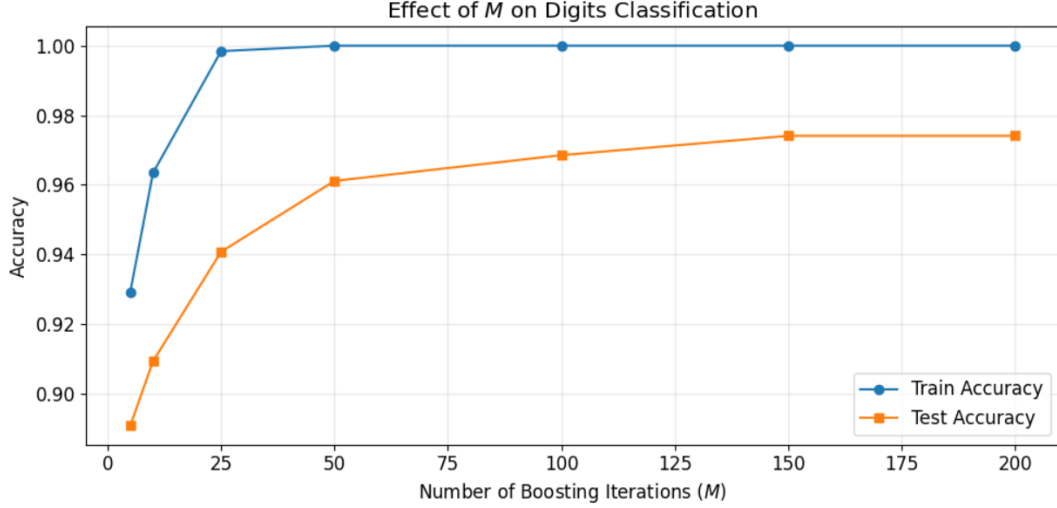


Figure 7: Effect of M on Digits classification. Test accuracy plateaus at $M \approx 150$ despite perfect training accuracy.

5.2 Tree Depth J

Table 5: Effect of tree depth on California Housing ($M=200$, $\nu=0.1$).

Depth	Train MSE	Test MSE	Gap
1	0.3949	0.4222	0.027
2	0.2694	0.3029	0.034
3	0.2194	0.2615	0.042
4	0.1712	0.2373	0.066
5	0.1294	0.2229	0.094
6	0.0897	0.2142	0.125
8	0.0318	0.2087	0.177

Key observation: Deeper trees improve test performance but cause a rapidly expanding train–test gap. At depth 8, train MSE is just 0.0318 (near-zero) while test MSE plateaus at 0.2087 ; the model is massively overfitting. The test MSE curve is nearly flat from depth 5 onward, suggesting that **depth 4–5 offers the best bias–variance tradeoff**. Stumps (depth 1) produce an additive model with no interactions, which is insufficient for this dataset.

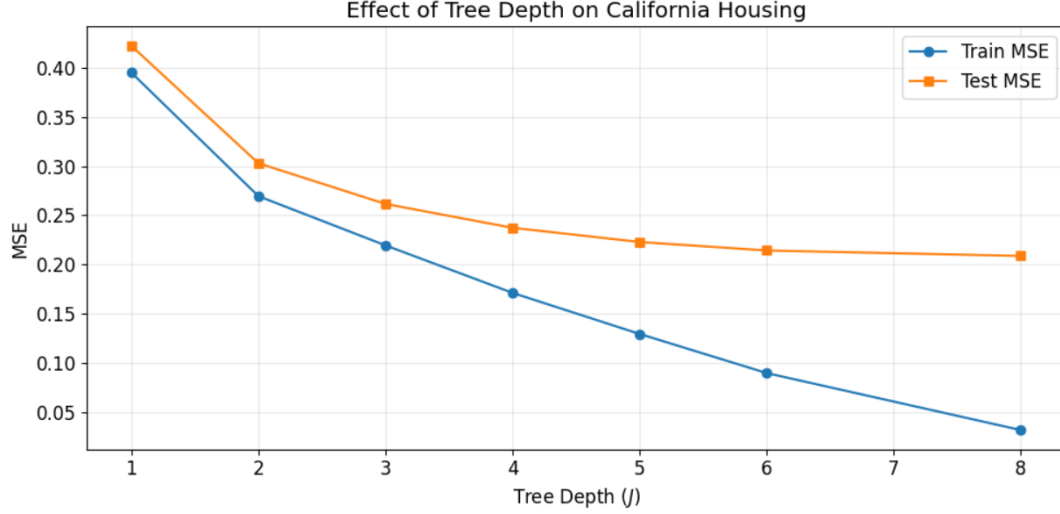


Figure 8: Effect of tree depth on California Housing. Train MSE drops to near-zero at depth 8, but test MSE flattens after depth 5.

5.3 Learning Rate ν (Shrinkage)

Key observation (Friedman #1, $M=300$): Smaller ν values produce slower convergence on the training loss but tend to achieve lower test MSE given enough iterations. Conversely, $\nu=1.0$ causes the model to oscillate and overfit aggressively; the test MSE increases after a few dozen iterations. This confirms the tradeoff identified in ESL: reducing ν by a factor of 10 roughly requires $10\times$ more iterations, but the resulting model generalizes better. A learning rate of $\nu \in [0.05, 0.1]$ provides a good balance.

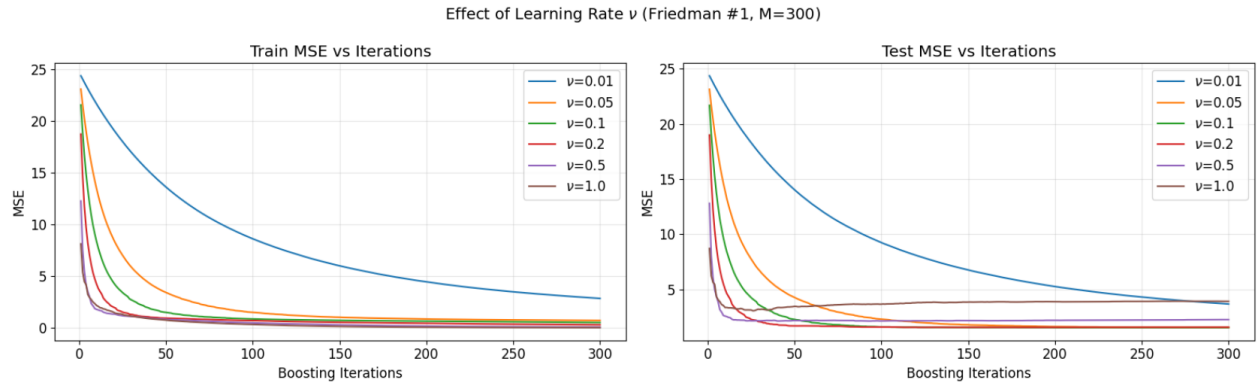


Figure 9: Effect of learning rate ν on Friedman #1. Left: train MSE. Right: test MSE. Smaller ν converges slower but achieves lower test error. $\nu=1.0$ overfits rapidly.

5.4 Loss Function under Outliers

We injected 5% outliers (200 samples with $\sigma=30$ Gaussian noise) into the Friedman #1 training data and compared all three loss functions.

Table 6: Loss function comparison with 5% outliers in training data ($M=200$, depth=4, $\nu=0.1$).

Loss	Test MSE	Test MAE
Squared Error	7.6816	1.7536
Absolute Error	1.7024	1.0317
Huber ($\alpha=0.9$)	1.8476	1.0619

Key observation: Squared error is severely degraded by outliers and its test MSE is $4.5\times$ worse than absolute error. The squared loss aggressively chases outlier residuals (since $\partial L/\partial f = -(y - f)$ grows linearly), causing the model to distort its predictions for inliers. Absolute error and Huber loss are robust because their gradients are bounded (± 1 and $\pm \delta_m$ respectively), preventing individual outliers from dominating the fit.

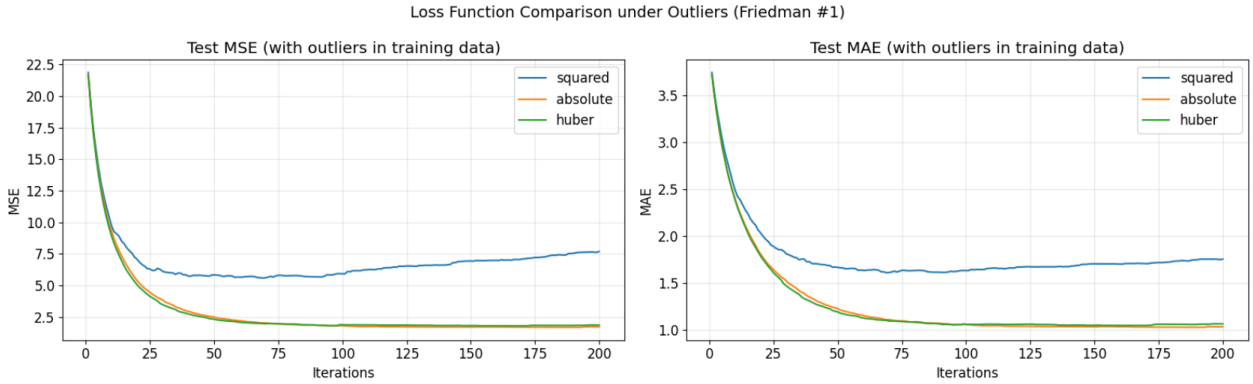


Figure 10: Loss function comparison under 5% outlier contamination (Friedman #1). Squared error is heavily degraded; absolute and Huber losses remain robust.

5.5 Depth $\times$ Learning Rate Interaction (Friedman #1)

Table 7: Best test MSE over 300 iterations for each (depth, ν) combination.

Depth $\setminus \nu$	0.01	0.05	0.10	0.20
1	11.30	3.94	3.14	3.10
2	6.96	2.31	1.60	1.57
3	4.75	1.72	1.50	1.55
4	3.68	1.55	1.54	1.56
6	2.46	1.63	1.64	1.77

Key observation: The best configuration is **depth 3**, $\nu=0.1$ (MSE 1.50). At small learning rates ($\nu=0.01$), 300 iterations are insufficient regardless of depth, the model is underfitting. At high depths (≥ 6) with moderate-to-large ν , performance degrades due to overfitting. This confirms that depth and learning rate must be tuned jointly; increasing one without adjusting the other leads to suboptimal results.

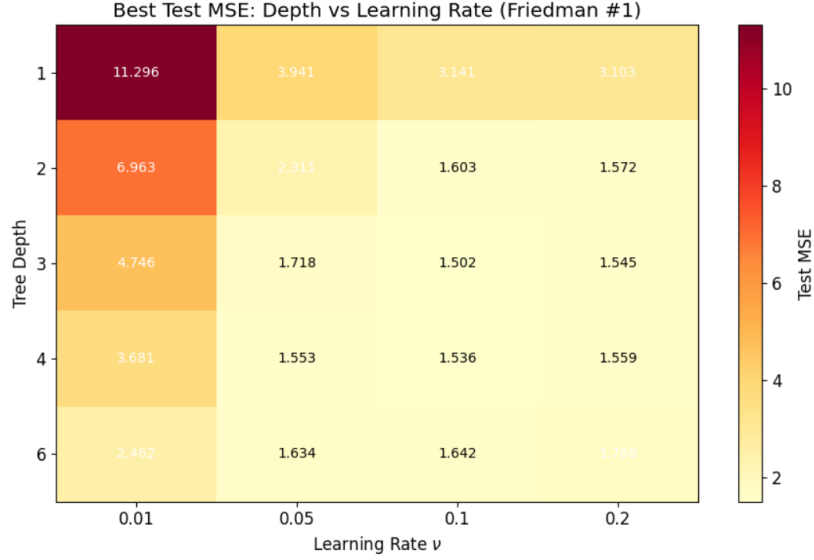


Figure 11: Best test MSE heatmap: depth vs learning rate (Friedman #1, $M=300$). The optimum lies at depth 3, $\nu=0.1$.

6 Comparison with Baselines

6.1 Regression — California Housing

Table 8: Baseline comparison for regression.

Model	Test MSE	Test R^2
Single Stump (depth=1)	0.9441	0.2795
Single Deep Tree (unlimited depth)	0.4958	0.6216
Boosted Stumps ($M=300$, depth=1)	0.3953	0.6983
Boosted Trees ($M=300$, depth=4)	0.2279	0.8261

Boosted trees with depth 4 achieve an R^2 of 0.83, compared to 0.28 for a single stump and 0.62 for a fully-grown tree. Even boosted stumps (0.70) outperform the deep tree, demonstrating the core thesis: an ensemble of weak learners with stagewise fitting generalizes better than one complex model.

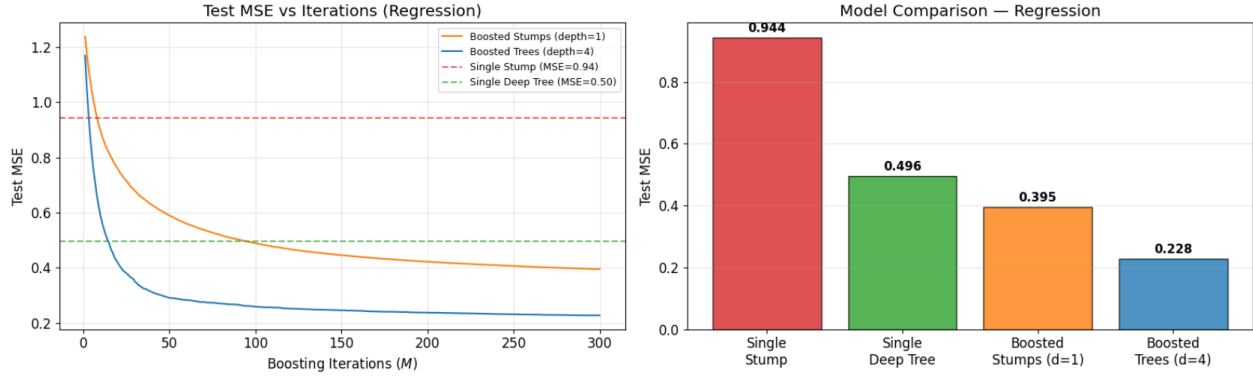


Figure 12: Regression baselines (California Housing). Left: test MSE vs boosting iterations with single-tree baselines as horizontal lines. Right: bar chart comparison of all four models.

6.2 Classification — Digits ($K=10$)

Table 9: Baseline comparison for classification.

Model	Test Accuracy	Test Error
Single Stump (depth=1)	0.1870	0.8130
Single Deep Tree (unlimited depth)	0.8333	0.1667
Boosted Stumps ($M=200$, depth=1)	0.9463	0.0537
Boosted Trees ($M=200$, depth=3)	0.9741	0.0259

The contrast is striking on this 10-class problem. A single stump can only split on one feature, achieving near-random performance (18.70%). Boosting 200 such stumps yields 94.63%, a **5 $\times$ reduction in error** over the deep tree. Boosted depth-3 trees push to 97.41%, reducing the error to just 2.59%.

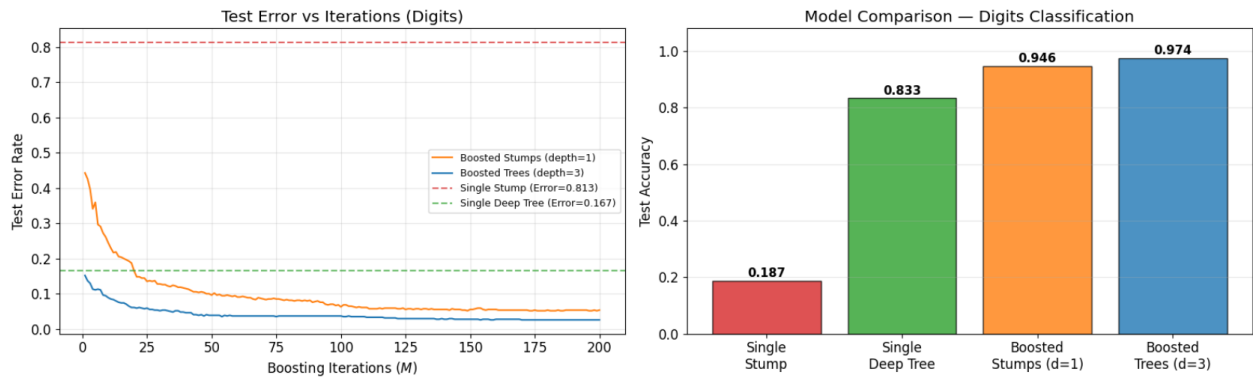


Figure 13: Classification baselines (Digits). Left: test error vs boosting iterations. Right: accuracy bar chart. The gap between single stump (18.7%) and boosted trees (97.4%) demonstrates boosting's power.

7 Training Dynamics

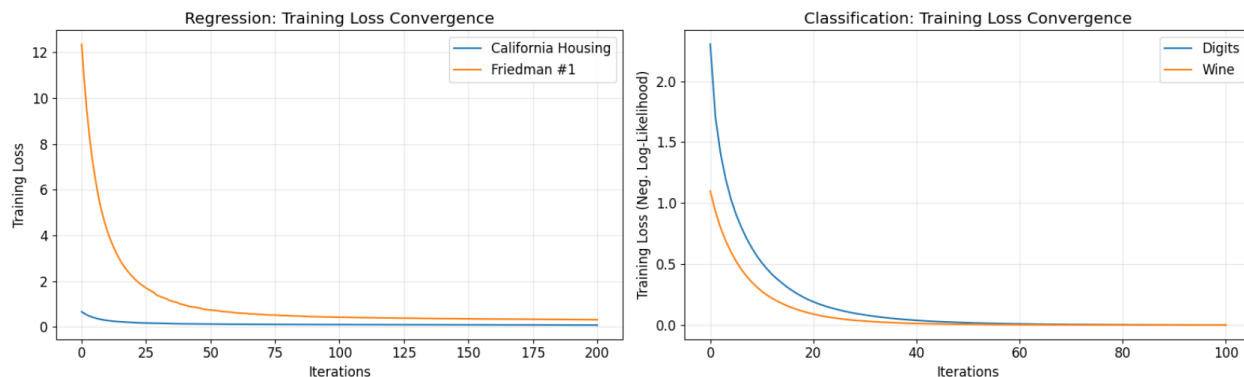


Figure 14: Training loss convergence. Left: regression (squared error). Right: classification (negative log-likelihood). Both show monotonic decrease, with Digits converging slower due to the 10-class problem.

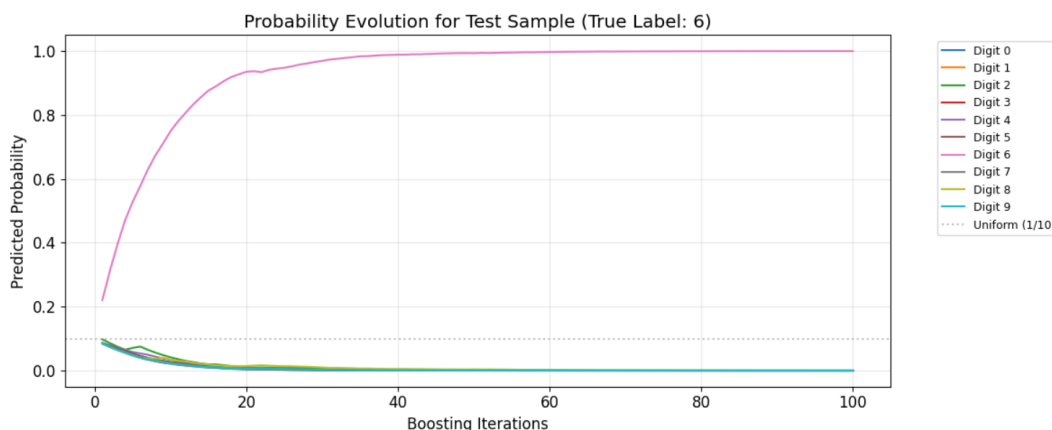


Figure 15: Predicted class probabilities across boosting iterations for a single Digits test sample. Starting from uniform $1/10$, the correct class probability rises sharply while others decay.

The probability evolution plot illustrates the mechanism of Algorithm 10.4: at each iteration, the softmax outputs are pushed toward the correct class by the additive updates. The initial uniform distribution ($1/K = 0.1$) resolves within ~ 20 iterations for well-separated examples, while harder examples require more rounds.

8 Discussion

- **Overfitting behavior.** Across all ablations, the train-test gap is the clearest indicator of overfitting. Increasing M , depth, or ν independently all widen this gap. The training loss curves show monotonic decrease, but the test curves flatten or rise and this is particularly visible in the learning rate experiment where $\nu=1.0$ causes rapid test-error degradation after ~ 50 iterations.

- **Shrinkage as regularization.** Reducing ν provides implicit regularization by limiting each tree’s influence. The shrinkage slide (ESL Figure 10.41) predicts that smaller ν with larger M yields better generalization, and our experiments confirm this: $\nu=0.05$ with 300 iterations consistently produces lower test error than $\nu=0.2$ with the same budget.
- **Robust losses.** The $4.5\times$ MSE reduction from squared to absolute error under 5% outliers is among the most practically important results. In real-world regression problems where outliers are common, the choice of loss function matters more than tuning M or depth.
- **Classification algorithm.** The $\frac{K-1}{K}$ factor in Algorithm 10.4’s leaf computation comes from approximating the full $K \times K$ Hessian with its diagonal, then correcting for the sum-to-zero constraint on the K class functions. Without this factor, the step sizes are slightly too large, which can cause overshooting in early iterations.
- **Single tree vs. ensemble.** The Digits experiment most clearly demonstrates boosting’s value. A single tree with unlimited depth overfits heavily (train accuracy 1.0, test 83.33%), while boosted depth-3 trees achieve 97.41% test accuracy with the same training-set memorization. The ensemble’s stagewise construction provides a form of regularization that a single greedy tree lacks.

9 Implementation Notes

- We use `sklearn.tree.DecisionTreeRegressor` as the base learner only. The boosting loop, gradient computation, leaf-value optimization (including the Newton step for classification), softmax, and shrinkage-based update are all implemented from scratch in NumPy.
- The classifier fits K separate trees per boosting iteration (one per class), each targeting the residuals $r_{ikm} = y_{ik} - p_k(x_i)$.
- Loss functions are implemented as modular classes with a common interface (`negative_gradient`, `optimal_leaf_value`), allowing easy extension.
- `staged_predict` generators enable efficient computation of learning curves without re-fitting models.

10 Conclusion

We implemented Algorithms 10.3 and 10.4 from scratch and validated them on four datasets. The key findings are:

1. Gradient boosting significantly outperforms single trees on both regression (R^2 : 0.83 vs. 0.62) and classification (error: 2.6% vs. 16.7%).
2. Hyperparameters interact: the optimal configuration for Friedman #1 is depth 3 with $\nu=0.1$; increasing either alone beyond this point degrades generalization.
3. Robust loss functions (absolute, Huber) are essential under outlier contamination, reducing test MSE by $4\text{--}5\times$ compared to squared error.
4. Shrinkage is the most effective regularizer: smaller ν with more iterations consistently improves test performance.